

间复杂度均为 $O(n^2)$ 。从 $2^n \sim n^2$ 是一个巨大的优化,这正是利用了数字三角形具有大量重叠子问题的特点。

提示 9-4: 可以用记忆化搜索的方法计算状态转移方程。当采用记忆化搜索时,不必事先确定各状态的计算顺序,但需要记录每个状态“是否已经计算过”。

9.2 DAG 上的动态规划

有向无环图上的动态规划是学习动态规划的基础。很多问题都可以转化为 DAG 上的最长路、最短路或路径计数问题。

9.2.1 DAG 模型

嵌套矩形问题。有 n 个矩形,每个矩形可以用两个整数 a, b 描述,表示它的长和宽。矩形 $X(a, b)$ 可以嵌套在矩形 $Y(c, d)$ 中,当且仅当 $a < c, b < d$, 或者 $b < c, a < d$ (相当于把矩形 X 旋转 90°)。例如, $(1, 5)$ 可以嵌套在 $(6, 2)$ 内,但不能嵌套在 $(3, 4)$ 内。你的任务是选出尽量多的矩形排成一行,使得除了最后一个之外,每一个矩形都可以嵌套在下一个矩形内。如果有多解,矩形编号的字典序应尽量小。

【分析】

矩形之间的“可嵌套”关系是一个典型的二元关系,二元关系可以用图来建模。如果矩形 X 可以嵌套在矩形 Y 里,就从 X 到 Y 连一条有向边。这个有向图是无环的,因为一个矩形无法直接或间接地嵌套在自己内部。换句话说,它是一个 DAG。这样,所要求的便是 DAG 上的最长路径。

硬币问题。有 n 种硬币,面值分别为 $V_1, V_2, \dots, V_n$, 每种都有无限多。给定非负整数 S , 可以选用多少个硬币,使得面值之和恰好为 S ? 输出硬币数目的最小值和最大值。 $1 \leq n \leq 100, 0 \leq S \leq 10000, 1 \leq V_i \leq S$ 。

【分析】

此问题尽管看上去和嵌套矩形问题很不一样,但本题的本质也是 DAG 上的路径问题。将每种面值看作一个点,表示“还需要凑足的面值”,则初始状态为 S , 目标状态为 0 。若当前在状态 i , 每使用一个硬币 j , 状态便转移到 $i - V_j$ 。

这个模型和上一题类似,但也有一些明显的不同之处:上题并没有确定路径的起点和终点(可以把任意矩形放在第一个和最后一个),而本题的起点必须为 S , 终点必须为 0 ; 点固定之后“最短路”才是有意义的。在上题中,最短序列显然是空(如果不允许空,就是单个矩形,不管怎样都是平凡的),而本题的最短路却不容易确定。

9.2.2 最长路及其字典序

首先思考“嵌套矩形”。如何求 DAG 中不固定起点的最长路径呢? 仿照数字三角形的

做法, 设 $d(i)$ 表示从结点 i 出发的最长路长度, 应该如何写状态转移方程呢? 第一步只能走到它的相邻点, 因此:

$$d(i) = \max\{d(j) + 1 \mid (i, j) \in E\}$$

其中, E 为边集。最终答案是所有 $d(i)$ 中的最大值。根据前面的介绍, 可以尝试按照递归或记忆化搜索的方式计算上式。不管怎样, 都需要先把图建立出来, 假设用邻接矩阵保存在矩阵 G 中 (在编写主程序之前需测试和调试程序, 以确保建图过程正确无误)。接下来编写记忆化搜索程序 (调用前需初始化 d 数组的所有值为 0):

```
int dp(int i) {
    int& ans = d[i];
    if(ans > 0) return ans;
    ans = 1;
    for(int j = 1; j <= n; j++)
        if(G[i][j]) ans = max(ans, dp(j)+1);
    return ans;
}
```

这里用到了一个技巧: 为表项 $d[i]$ 声明一个引用 ans 。这样, 任何对 ans 的读写实际上都是在对 $d[i]$ 进行。当 $d[i]$ 换成 $d[i][j][k][l][m][n]$ 这样很长的名字时, 该技巧的优势就会很明显。

提示 9-5: 在记忆化搜索中, 可以为正在处理的表项声明一个引用, 简化对它的读写操作。

原题还有一个要求: 如果有多个最优解, 矩形编号的字典序应最小。还记得第 6 章中的例题“理想路径”吗? 方法与其类似。将所有 d 值计算出来以后, 选择最大 $d[i]$ 所对应的 i 。如果有多个 i , 则选择最小的 i , 这样才能保证字典序最小。接下来可以选择 $d(i) = d(j) + 1$ 且 $(i, j) \in E$ 的任何一个 j 。为了让方案的字典序最小, 应选择其中最小的 j 。程序如下^①:

```
void print_ans(int i) {
    printf("%d ", i);
    for(int j = 1; j <= n; j++) if(G[i][j] && d[i] == d[j]+1){
        print_ans(j);
        break;
    }
}
```

提示 9-6: 根据各个状态的指标值可以依次确定各个最优决策, 从而构造出完整方案。由于决策是依次确定的, 所以很容易按照字典序打印出所有方案。

注意, 当找到一个满足 $d[i] = d[j] + 1$ 的结点 j 后就应立刻递归打印从 j 开始的路径, 并在递归返回后退出循环。如果要打印所有方案, 只把 `break` 语句删除是不够的 (想一想, 为什么)。正确的方法是记录路径上的所有点, 在递归结束时才一次性输出整条路径。程序

^① 输出的最后会有一个多余空格, 并且没有回车符。在使用时, 应在主程序调用 `print_ans` 后加一个回车符。如果比赛明确规定行末不允许有多余空格, 则可以像前面介绍的那样加一个变量 `first` 来帮助判断。

留给读者编写。

有趣的是，如果把状态定义成“ $d(i)$ 表示以结点 i 为终点的最长路径长度”，也能顺利求出最优值，却难以打印出字典序最小的方案。想一想，为什么？你能总结出一些规律吗？

9.2.3 固定终点的最长路和最短路

接下来考虑“硬币问题”。最长路和最短路的求法是类似的，下面只考虑最长路。由于终点固定， $d(i)$ 的确切含义变为“从结点 i 出发到结点 0 的最长路径长度”。下面是求最长路的代码：

```
int dp(int S) {
    int& ans = d[S];
    if(ans >= 0) return ans;
    ans = 0;
    for(int i = 1; i <= n; i++) if(S >= V[i]) ans = max(ans, dp(S-V[i])+1);
    return ans;
}
```

注意到区别了吗？由于在本题中，路径长度是可以为 0 的（ S 本身可以是 0），所以不能再用 $d=0$ 表示“这个 d 值还没有算过”。相应地，初始化时也不能再把 d 全设为 0，而要设置为一个负值——在正常情况下是取不到的。常见的方法是用 -1 来表示“没有算过”，则初始化时只需用 `memset(d, -1, sizeof(d))` 即可。至此，已完整解释了上面的代码为什么把 `if(ans>0)` 改成了 `if(ans>=0)`。

提示 9-7：当程序中需要用到特殊值时，应确保该值在正常情况下不会被取到。这不仅意味着特殊值不能有“正常的理解方式”，而且也不能在正常运算中“意外得到”。

不知读者有没有看出，上述代码有一个致命的错误，即由于结点 S 不一定真的能到达结点 0，所以需要特殊的 $d[S]$ 值表示“无法到达”，但在上述代码中，如果 S 根本无法继续往前走，返回值是 0，将被误以为是“不用走，已经到达终点”的意思。如果把 `ans` 初始化为 -1 呢？别忘了 -1 代表“还没算过”，所以返回 -1 相当于放弃了自己的劳动成果。如果把 `ans` 初始化为一个很大的整数，例如 2^{30} 呢？如果一开始就这么大，`ans = max(ans, dp(i)+1)` 还能把 `ans` 变回“正常值”吗？如果改成很小的整数，例如 -2^{30} 呢？从目前来看，它也会被认为是“还没算过”，但至少可以和所有 d 的初值分开——只需把代码中 `if(ans>=0)` 改为 `if(ans!= -1)` 即可，如下所示：

```
int dp(int S){
    int& ans = d[S];
    if(ans != -1) return ans;
    ans = -(1<<30);
    for(int i = 1; i <= n; i++) if(S >= V[i]) ans = max(ans, dp(S-V[i])+1);
    return ans;
}
```

提示 9-8: 在记忆化搜索中, 如果用特殊值表示“还没算过”, 则必须将其和其他特殊值(如无解)区分开。

上述错误都是很常见的, 甚至“顶尖高手”有时也会一时糊涂, 掉入陷阱。意识到这些问题, 寻求解决方案是不难的, 但就怕调试很久以后仍然没有发现哪里出了问题。另一个解决方法是不用特殊值表示“还没算过”, 而用另外一个数组 `vis[i]` 表示状态 `i` 是否被访问过, 如下所示:

```
int dp(int S){
    if(vis[S]) return d[S];
    vis[S] = 1;
    int& ans = d[S];
    ans = -(1<<30);
    for(int i = 1; i <= n; i++) if(S >= V[i]) ans = max(ans, dp(S-V[i])+1);
    return ans;
}
```

尽管多了一个数组, 但可读性增强了许多: 再也不用担心特殊值之间的冲突了, 在任何情况下, 记忆化搜索的初始化都可以用 `memset(vis, 0, sizeof(vis))`^①实现。

提示 9-9: 在记忆化搜索中, 可以用 `vis` 数组记录每个状态是否计算过, 以占用一些内存为代价增强程序的可读性, 同时减少出错的可能。

本题要求最小、最大两个值, 记忆化搜索就必须写两个。在这种情况下, 用递推更加方便(此时需注意递推的顺序):

```
minv[0] = maxv[0] = 0;
for(int i = 1; i <= S; i++){
    minv[i] = INF; maxv[i] = -INF;
}
for(int i = 1; i <= S; i++)
    for(int j = 1; j <= n; j++)
        if(i >= V[j]){
            minv[i] = min(minv[i], minv[i-V[j]] + 1);
            maxv[i] = max(maxv[i], maxv[i-V[j]] + 1);
        }
printf("%d %d\n", minv[S], maxv[S]);
```

如何输出字典序最小的方案呢? 刚刚介绍的方法仍然适用, 如下所示:

```
void print_ans(int* d, int S){
    for(int i = 1; i <= n; i++)
        if(S >= V[i] && d[S] == d[S-V[i]]+1){
```

^① 如果状态比较复杂, 推荐用 STL 中的 `map` 而不是普通数组保存状态值。这样, 判断状态 `S` 是否算过只需用 `if(d.count(S))` 即可。

```
printf("%d ", i);
print_ans(d, S-V[i]);
break;
}
}
```

然后分别调用 `print_ans(min, S)` (注意在后面要加一个回车符) 和 `print_ans(max, S)` 即可。输出路径部分和上题的区别是，上题打印的是路径上的点，而这里打印的是路径上的边。还记得数组可以作为指针传递吗？这里需要强调的一点是：数组作为指针传递时，不会复制数组中的数据，因此不必担心这样会带来不必要的时间开销。

提示 9-10：当用递推法计算出各个状态的指标之后，可以用与记忆化搜索完全相同的方式打印方案。

很多用户喜欢另外一种打印路径的方法：递推时直接用 `min_coin[S]` 记录满足 $\min[S] = \min[S-V[i]]+1$ 的最小的 i ，则打印路径时可以省去 `print_ans` 函数中的循环，并可以方便地把递归改成迭代（原来的也可以改成迭代，但不那么自然）。具体来说，需要把递推过程改成以下形式：

```
for(int i = 1; i <= S; i++)
for(int j = 1; j <= n; j++)
if(i >= V[j]){
    if(min[i] > min[i-V[j]] + 1){
        min[i] = min[i-V[j]] + 1;
        min_coin[i] = j;
    }
    if(max[i] < max[i-V[j]] + 1){
        max[i] = max[i-V[j]] + 1;
        max_coin[i] = j;
    }
}
```

注意，判断中用的是“>”和“<”，而不是“>=”和“<=”，原因在于“字典序最小解”要求当 `min/max` 值相同时取最小的 i 值。反过来，如果 j 是从大到小枚举的，就需要把“>”和“<”改成“>=”和“<=”才能求出字典序最小解。

在求出 `min_coin` 和 `max_coin` 之后，只需调用 `print_ans(min_coin, S)` 和 `print_ans(max_coin, S)` 即可。

```
void print_ans(int* d, int S){
    while(S){
        printf("%d ", d[S]);
        S -= V[d[S]];
    }
}
```

该方法是一个“用空间换时间”的经典例子——用 `min_coin` 和 `max_coin` 数组消除了原来 `print_ans` 中的循环。

提示 9-11: 无论是用记忆化搜索还是递推，如果在计算最优值的同时“顺便”算出各个状态下的第一次最优决策，则往往能让打印方案的过程更加简单、高效。这是一个典型的“用空间换时间”的例子。

9.2.4 小结与应用举例

本节介绍了动态规划的经典应用：DAG 中的最长路和最短路。和 9.1 节中的数字三角形问题一样，DAG 的最长路和最短路都可以用记忆化搜索和递推两种实现方式。打印解时既可以根据 `d` 值重新计算出每一步的最优决策，也可以在动态规划时“顺便”记录下每一步的最优决策。

由于 DAG 最长（短）路的特殊性，有两种“对称”的状态定义方式。

状态 1：设 $d(i)$ 为从 i 出发的最长路，则 $d(i) = \max\{d(j) + 1 \mid (i, j) \in E\}$ 。

状态 2：设 $d(i)$ 为以 i 结束的最长路，则 $d(i) = \max\{d(j) + 1 \mid (j, i) \in E\}$ 。

如果使用状态 2，“硬币问题”就变得和“嵌套矩形问题”几乎一样了（唯一的区别是：“嵌套矩形问题”还需要取所有 $d(i)$ 的最大值）！9.2.3 节中有意介绍了比较麻烦的状态 1，主要是为了展示一些常见技巧和陷阱，实际比赛中不推荐使用。

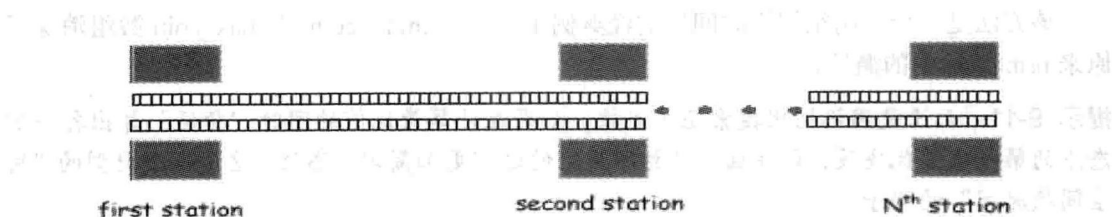
使用状态 2 时，有时还会遇到一个问题：状态转移方程可能不好计算，因为在很多时候，可以方便地枚举从某个结点 i 出发的所有边 (i, j) ，却不方便“反着”枚举 (j, i) 。特别是在有些题目中，这些边具有明显的实际背景，对应的过程不可逆。

这时需要用“刷表法”。什么是“刷表法”呢？传统的递推法可以表示成“对于每个状态 i ，计算 $f(i)$ ”，或者称为“填表法”。这需要对于每个状态 i ，找到 $f(i)$ 依赖的所有状态，在某些情况下并不方便。另一种方法是“对于每个状态 i ，更新 $f(i)$ 所影响到的状态”，或者称为“刷表法”。对应到 DAG 最长路的问题中，就相当于按照拓扑序枚举 i ，对于每个 i ，枚举边 (i, j) ，然后更新 $d[j] = \max(d[j], d[i] + 1)$ 。注意，一般不把这个式子叫做“状态转移方程”，因为它不是一个可以直接计算 $d[j]$ 的方程，而只是一个更新公式。

提示 9-12: 传统的递推法可以表示成“对于每个状态 i ，计算 $f(i)$ ”，或者称为“填表法”。这需要对于每个状态 i ，找到 $f(i)$ 依赖的所有状态，在某些时候并不方便。另一种方法是“对于每个状态 i ，更新 $f(i)$ 所影响到的状态”，或者称为“刷表法”，有时比填表法方便。但需要注意的是，只有当每个状态所依赖的状态对它的影响相互独立时才能用刷表法。

例题 9-1 城市里的间谍 (A Spy in the Metro, ACM/ICPC World Finals 2003, UVa1025)

某城市的地铁是线性的，有 n ($2 \leq n \leq 50$) 个车站，从左到右编号为 $1 \sim n$ 。有 $M1$ 辆列车从第 1 站开始往右开，还有 $M2$ 辆列车从第 n 站开始往左开。在时刻 0，Mario 从第 1 站出发，目的是在时刻 T ($0 \leq T \leq 200$) 会见车站 n 的一个间谍。在车站等车时容易被抓，所以她决定尽量躲在开动的火车上，让在车站等待的总时间尽量短。列车靠站停车时间忽略不计，且 Mario 身手敏捷，即使两辆方向不同的列车在同一时间靠站，Mario 也能完成换乘。



输入第 1 行为 n ，第 2 行为 T ，第 3 行有 $n-1$ 个整数 $t_1, t_2, \dots, t_{n-1}$ ($1 \leq t_i \leq 70$)，其中 t_i 表示地铁从车站 i 到 $i+1$ 的行驶时间（两个方向一样）。第 4 行为 M_1 ($1 \leq M_1 \leq 50$)，即从第 1 站出发向右开的列车数目。第 5 行包含 M_1 个整数 $d_1, d_2, \dots, d_{M_1}$ ($0 \leq d_i \leq 250, d_i < d_{i+1}$)，即各列车的出发时间。第 6、7 行描述从第 n 站出发向左开的列车，格式同第 4、5 行。输出仅包含一行，即最少等待时间。无解输出 impossible。

【分析】

时间是单向流逝的，是一个天然的“序”。影响到决策的只有当前时间和所处的车站，所以可以用 $d(i, j)$ 表示时刻 i ，你在车站 j （编号为 $1 \sim n$ ），最少还需要等待多长时间。边界条件是 $d(T, n) = 0$ ，其他 $d(T, i)$ (i 不等于 n) 为正无穷。有如下 3 种决策。

决策 1：等 1 分钟。

决策 2：搭乘往右开的车（如果有）。

决策 3：搭乘往左开的车（如果有）。

主过程的代码如下：

```
for(int i = 1; i <= n-1; i++) dp[T][i] = INF;
dp[T][n] = 0;

for(int i = T-1; i >= 0; i--)
    for(int j = 1; j <= n; j++) {
        dp[i][j] = dp[i+1][j] + 1; //等待一个单位
        if(j < n && has_train[i][j][0] && i+t[j] <= T)
            dp[i][j] = min(dp[i][j], dp[i+t[j]][j+1]); //右
        if(j > 1 && has_train[i][j][1] && i+t[j-1] <= T)
            dp[i][j] = min(dp[i][j], dp[i+t[j-1]][j-1]); //左
    }

//输出
cout << "Case Number " << ++kase << ": ";
if(dp[0][1] >= INF) cout << "impossible\n";
else cout << dp[0][1] << "\n";
```

上面的代码中有一个 `has_train` 数组，其中 `has_train[t][i][0]` 表示时刻 t ，在车站 i 是否有往右开的火车，`has_train[t][i][1]` 类似，不过记录的是往左开的火车。这个数组不难在输入时计算处理，细节留给读者思考。

状态有 $O(nT)$ 个，每个状态最多只有 3 个决策，因此总时间复杂度为 $O(nT)$ 。

例题 9-2 巴比伦塔 (The Tower of Babylon, UVa 437)

有 n ($n \leq 30$) 种立方体, 每种都有无穷多个。要求选一些立方体摞成一根尽量高的柱子 (可以自行选择哪一条边作为高), 使得每个立方体的底面长宽分别严格小于它下方立方体的底面长宽。

【分析】

在任何时候, 只有顶面的尺寸会影响到后续决策, 因此可以用二元组 (a, b) 来表示“顶面尺寸为 $a \times b$ ”这个状态。因为每次增加一个立方体以后顶面的长和宽都会严格减小, 所以这个图是 DAG, 可以套用前面学过的 DAG 最长路算法。

这个算法没问题, 不过落实到程序上时会遇到一个问题: 不能直接用 $d(a, b)$ 表示状态值, 因为 a 和 b 可能会很大。怎么办呢? 可以用 (idx, k) 这个二元组来“间接”表达这个状态, 其中 idx 为顶面立方体的序号, k 是高的序号 (假设输入时把每个立方体的 3 个维度从小到大排序, 编号为 0~2)。例如, 若立方体 3 的大小为 $a \times b \times c$ (其中 $a \leq b \leq c$), 则状态 $(3, 1)$ 就是指这个立方体在顶面, 且高是 b (因此顶面大小为 $a \times c$)。因为 idx 是 $0 \sim n-1$ 的整数, k 是 $0 \sim 2$ 的整数, 所以可以很方便地用二维数组来存取。状态总数是 $O(n)$ 的, 每个状态的决策有 $O(n)$ 个, 时间复杂度为 $O(n^2)$ 。

例题 9-3 旅行 (Tour, ACM/ICPC SEERC 2005, UVa1347)

给定平面上 n ($n \leq 1000$) 个点的坐标 (按照 x 递增的顺序给出。各点 x 坐标不同, 且均为正整数), 你的任务是设计一条路线, 从最左边的点出发, 走到最右边的点后再返回, 要求除了最左点和最右点之外每个点恰好经过一次, 且路径总长度最短。两点间的长度为它们的欧几里德距离, 如图 9-4 所示。

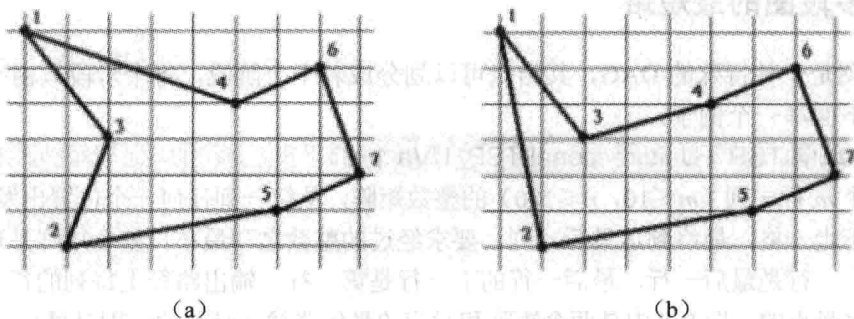


图 9-4 旅行路线示意图

【分析】

“从左到右再回来”不太方便思考, 可以改成: 两个人同时从最左点出发, 沿着两条不同的路径走, 最后都走到最右点, 且除了起点和终点外其余每个点恰好被一个人经过。这样, 就可以用 $d(i, j)$ 表示第一个人走到 i , 第二个人走到 j , 还需要走多长的距离。

状态如何转移呢? 仔细思考后会发现: 好像很难保证两个人不会走到相同的点。例如, 计算状态 $d(i, j)$ 时, 能不能让 i 走到 $i+1$ 呢? 不知道, 因为从状态里看不出来 $i+1$ 有没有被 j 走过。换句话说, 状态定义得不好, 导致转移困难。

下面修改一下: $d(i, j)$ 表示 $1 \sim \max(i, j)$ 全部走过, 且两个人的当前位置分别是 i 和 j , 还需